Phase 2: Requirement Analysis — Data Flow Diagrams (DFD)

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Level 0 Context DFD (System Boundary)

The Level 0 Context DFD illustrates the high-level data interactions between the external entities (Urban Rider and Executive Dispatch Admin) and the centralized UCab Full-Stack System.

```
graph LR
Rider([Urban Rider / User])
Admin([Executive Dispatch Admin])
System((UCab Full-Stack MERN System))
Rider -->|Registration / Login Credentials| System
Rider -->|Trip Booking Request & Perks Selection| System
Rider -->|Promo Code & Payment Confirmation| System
System -->|JWT Session Token & User Profile| Rider
System -->|Fleet Inventory & Fare Breakdown| Rider
System -->|Live Dispatch Tracking Status & PDF Invoice| Rider
Admin -->|Admin Auth Credentials| System
Admin -->|Fleet Inventory CRUD (Add/Edit Cab)| System
Admin -->|Ride Dispatch Control (Update Trip Status)| System
System -->|Analytics KPI Metrics & Gross Revenue| Admin
System -->|Registered User Directory & Trip Logs| Admin
```

2. Level 1 DFD (Trip Booking & Dispatch Process)

The Level 1 DFD decomposes the system into specific internal processes, data stores (`MongoDB / data.json`), and data flows during trip booking and executive dispatch.

```
graph TD
subgraph External Entities
User([Urban Rider])
Dispatch([Admin Dispatcher])
end
subgraph Internal Processes
P1(P1: Authentication & Token Verification)
P2(P2: Fleet Inventory & Fare Engine)
P3(P3: Trip Creation & Perks Calculator)
P4(P4: Dispatch Synchronization & Status Update)
end
subgraph Data Persistence Layer
D1[(D1: User / Admin Store)]
D2[(D2: Car Fleet Store)]
D3[(D3: Booking Trip Store)]
end
User -->|Email + Password / Demo Click| P1
P1 -->|Query Credentials| D1
D1 -->|User Record / Role| P1
P1 -->|Issue JWT Bearer Token| User
User -->|Fetch Cabs & Filter by Category| P2
P2 -->|Query Available Cabs| D2
D2 -->|Cab List & Rates (` ¹/km`)| P2
P2 -->|Return Sorted Cabs| User
```

```
User -->|Post Booking Request (Distance, Promos, Perks)| P3
P3 -->|Validate Promo (`UCAB20`) & Compute Total| P3
P3 -->|Save New Booking Record| D3
D3 -->|Trip Confirmation ID| P3
P3 -->|Redirect to Tracking Bar| User
Dispatch -->|Update Trip Status (`Accepted` -> `Started`)| P4
P4 -->|Modify Booking Status| D3
D3 -->|Updated Trip State| P4
P4 -->|Real-Time Status Sync| User
```

3. Level 2 DFD (Zero-Config Dual Database Engine)

As a solo developer, ensuring that the application works flawlessly across any grading machine required building an automatic dual-engine persistence adapter (`server/db/config.js` and `server/db/store.js`).

```
graph TD
API[Express.js REST API Controllers]
Check{Global Engine Check: global.isMongoConnected?}
subgraph Mongoose ORM Engine
M_Find[Mongoose .find() / .create()]
Mongo[(MongoDB Daemon Port 27017 / Atlas)]
end
subgraph JSON Fallback File Engine
J_Load[loadData() from atomic file]
J_Write[saveData() atomic write]
File[(server/db/data.json)]
end
API --> Check
Check -->|True: MongoDB Online| M_Find
M_Find --> Mongo
Mongo --> M_Find
Check -->|False: Offline/Isolated Demo| J_Load
Check -->|False: Offline/Isolated Demo| J_Write
J_Load --> File
J_Write --> File
File --> J_Load
```